AXIOM MANIFESTO

A Conceptual Framework for Agent-Native Commerce

on Kaspa + Igra

Version 1.0 | June 2026

Independent Research Project by Andreas (pay.igra)

"The Internet was built for humans.

The next economy will be built for agents."

Axiom Manifesto

A Conceptual Framework for Agent-Native Commerce on Kaspa + Igra

Independent research. Not an official roadmap, protocol specification or endorsement by Kaspa, Igra, Argent, Kaskad or any ecosystem entity.

1. Introduction

The Internet Was Built For Humans

The modern internet was designed around human cognition. Humans authenticate themselves. Humans read information. Humans authorize payments. Humans settle disputes.

The next wave of economic activity will increasingly be delegated to autonomous software agents.

These agents will not merely retrieve information. They will negotiate, coordinate, purchase, lend, borrow and settle value on behalf of individuals and organizations.

This shift requires more than faster blockchains.

It requires a coherent architecture for trust, authorization, truth, execution and settlement.

Axiom proposes a conceptual framework for discussing this emerging agent economy.

The Limitation

Current stacks rely on patching account-based chains (ERC-4337/7702), forcing agents into a congested, shared global state.

The Axiom Shift

We introduce a **covenant-native composable state** model, treating agents as actors within a decomposable state machine. This directly extends the technical vision pioneered by Kaskad — where composable state transitions enable parallel, non-interfering economic flows — into a complete semantic framework for agent-native commerce.

Where Kaskad provides the machinery for covenant-native composable state, Axiom provides the **economic vocabulary** that makes that machinery operable for autonomous agents.

The Outcome

By leveraging UTXO locality and atomic transitions, we enable high-frequency economic flows that are structurally hardcoded, not added as an afterthought.

2. Why Layers Matter

The history of computing shows that complex systems emerge through abstraction layers.

The internet evolved through:

- TCP/IP (transport)
- DNS (naming)
- HTTP (application protocol)
- Applications (user-facing)

Financial systems evolved through:

- Identity (who)
- Accounts (where)
- Payment networks (how)
- Settlement systems (when final)

Agent-native commerce may require similar abstraction layers.

Axiom organizes these capabilities into five conceptual layers.

3. The Five Layers

Layer 1 — Orchestration

Purpose: Agents need a way to express objectives and coordinate execution.

Primitives:

- `intent.igra` — Express economic objective
- `agents.igra` — Agent registry and discovery

- `mcp.igra` — Model Context Protocol for inter-agent communication

Technical Context:

Recent work such as Argent explores how complex workflows may be decomposed into multiple covenant transitions rather than executed inside a single monolithic contract. This suggests that future applications may increasingly resemble orchestrated flows of state transitions.

Kaskad's approach to **covenant-native composable state** demonstrates that parallel, non-interfering transitions are not merely possible — they are the native execution model for agent-native systems on Kaspa.

Intent becomes the starting point. Orchestration becomes the routing layer.

Layer 2 — Identity & Governance

Purpose: Agents must act on behalf of principals. Without verifiable authorization there is no accountable commerce.

Primitives:

- `auth.igra` — Scoped, revocable permissions
- `identity.igra` — Verifiable agent identity
- `governance.igra` — Mutable policy parameters

Technical Context:

Every economic action requires permission. Human owners need revocable authorization, scoped permissions and accountability.

Identity transforms autonomous behavior into delegated action.

Layer 3 — Data & Truth

Purpose: An agent is only as reliable as the information it acts upon.

Primitives:

- `data.igra` — Structured state schemas
- `oracle.igra` — External truth attestation
- `logs.igra` — Immutable audit trail

Technical Context:

Future agent systems require trusted data, signed observations and auditable decision trails.

The value of logs increases as autonomous systems become accountable for financial outcomes. Auditability becomes a first-class requirement rather than an afterthought.

Layer 4 — Economic Execution

Purpose: Agents require economic action. Not merely computation. Action.

Primitives:

- `pay.igra` — Value transfer
- `escrow.igra` — Conditional fund locking
- `settle.igra` — Final confirmation
- `supply.igra` — Liquidity provision
- `borrow.igra` — Collateralized debt
- `repay.igra` — Debt settlement

Technical Context:

Economic execution represents the transition from decision to value transfer.

Escrow introduces conditional commitment. Settlement introduces finality. Lending primitives enable persistent economic relationships between agents.

Layer 5 — Infrastructure

Purpose: Higher layers depend on shared infrastructure.

Primitives:

- `kskd.igra` — Native Kaspa denomination and fee handling
- `custody.igra` — Multi-sig and treasury management
- `kurrent.igra` — Real-time exchange rate and cross-chain settlement

Technical Context:

Infrastructure provides state synchronization, custody models and information transport.

These components are largely invisible to users but critical to system reliability.

4. The Agentic Pay Example

A simple workflow illustrates the interaction between layers.

User objective: Purchase a laptop below $1,200.

Flow:

```
intent → auth → data → escrow → pay → settle → logs
```

The agent:

1. Receives an objective

2. Verifies authorization

3. Retrieves trusted information (price, availability)

4. Reserves funds in escrow

5. Executes payment upon delivery confirmation

6. Finalizes settlement

7. Records the audit trail

The flow demonstrates how multiple primitives combine into a coherent economic action.

5. Axiom and Decomposable Computation

Recent research in the Kaspa ecosystem explores multi-contract architectures and decomposable computation.

Rather than executing all logic inside a single contract, applications may increasingly be expressed as sequences of covenant transitions — each verifying the next, each carrying its own state. Kaskad's work on **covenant-native composable state** provides the technical foundation for this paradigm.

Axiom does not attempt to specify these transitions.

Instead, it proposes a vocabulary for discussing the roles these transitions play in agent-native commerce. Where Kaskad builds the engine, Axiom builds the dashboard — the semantic layer that makes composable state economically meaningful.

The framework focuses on semantic structure rather than contract implementation.

6. Conclusion

The purpose of Axiom is not to predict the future roadmap of Kaspa or Igra.

Its purpose is to provide a common language for discussing agent-native commerce before the infrastructure fully exists.

Technology evolves. Shared vocabulary often emerges slowly.

Axiom is an attempt to map that vocabulary before the market demands it.

Appendix: The 17 Primitives

#	Primitive	Layer	Function
1	`intent.igra`	L1	Express economic goal
2	`agents.igra`	L1	Agent registry & discovery
3	`mcp.igra`	L1	Inter-agent communication
4	`auth.igra`	L2	Scoped permissions
5	`identity.igra`	L2	Verifiable agent ID
6	`governance.igra`	L2	Mutable policy
7	`data.igra`	L3	Structured state
8	`oracle.igra`	L3	External truth
9	`logs.igra`	L3	Audit trail
10	`pay.igra`	L4	Value transfer
11	`escrow.igra`	L4	Conditional locking
12	`settle.igra`	L4	Final confirmation
13	`supply.igra`	L4	Liquidity provision
14	`borrow.igra`	L4	Collateralized debt
15	`repay.igra`	L4	Debt settlement
16	`kskd.igra`	L5	Native fee handling

17	`custody.igra`	L5	Treasury management
18	`kurrent.igra`	L5	Real-time exchange

Built on Axioms. Designed for Autonomous Commerce.

The Internet was built for humans. The next economy will be built for agents.

PART II: EXTENDED EDITION

Technical Derivation of the Agent Economy Stack

The following sections extend the conceptual framework with deep technical mappings to Kaspa's UTXO layer, Argent DSL, and Silverscript primitives. This part is intended for builders, researchers, and investors seeking technical depth beyond the conceptual overview.

Axiom Manifesto — Extended Edition

Technical Derivation of the Agent Economy Stack

This document extends the conceptual framework with technical mappings to Kaspa's UTXO layer, Argent DSL, Silverscript primitives, and Kaskad's covenant-native composable state model.

0. Prologue

On June 18, 2026, Michael Sutton — founder of Kaspa — published a research prototype called **Argent**: a high-level DSL for multi-contract covenant applications that compile to Silverscript.

In doing so, he demonstrated something that this framework had hypothesized for months:

> ***Decomposable computation across multiple covenant contracts is not a theoretical curiosity. It is the native programming model for the Agent Economy.***

This document provides the conceptual architecture that Argent's compiler implements. Where Argent answers *how* agents compose contracts, Axiom answers *what* they compose, *why* the composition follows this structure, and *which* economic primitives emerge from it.

We do not claim ownership of Argent's technical achievements. We propose a **shared vocabulary** for discussing the roles these transitions play in agent-native commerce.

Acknowledgment to Kaskad: The term "covenant-native composable state" and its technical implementation are pioneered by the Kaskad team. Axiom extends this concept into the semantic and economic layer, proposing the vocabulary that makes composable state operable for autonomous agents.

1. The Hard Constraint: Why Kaspa, Why Now

1.1 The Four Requirements of Agentic Commerce

Requirement	PoS Chains	Bitcoin	Kaspa (Post-Toccata)
Low fees	Yes	No	Yes
Fast finality	Probabilistic	10 min	Sub-second
Programmability	Yes	Limited	Covenants + Argent
PoW security	No	Yes	Yes

No chain before Kaspa satisfied all four. After Toccata, Kaspa does.

1.2 Why UTXO > Account Model for Agents

Ethereum's account model treats smart contracts as permanent global objects in long-lived storage. This creates three problems for agents:

1. **State bloat:** Every possible execution path must exist in global state, even if never used.

2. **Front-running:** Global shared state is visible to MEV bots before confirmation.

3. **Composability cost:** Interacting with multiple contracts requires sequential transactions.

Kaspa's UTXO model inverts this:

> *"Complexity moves away from the permanent storage axis and onto the transient, prunable block-data axis. We don't pay by publishing every possible validator branch into permanent global state just to exist — we pay operationally when a path is actually used."* — Michael Sutton

For agents, this means:

- **Parallel execution:** Multiple covenant transitions can occur in the same block.
- **Local state:** Each UTXO carries its own state; no global lock contention.
- **Prunable history:** Old states disappear, reducing long-term storage costs.

1.3 BlockDAG as the Convergence Engine

Kaspa's GHOSTDAG protocol provides a critical property for multi-contract flows: **high-frequency block production with instant confirmation**. When an agent splits a computation across 32 UTXOs for parallel access, the time until those states converge is logarithmic in the number of parallel branches.

This is not an implementation detail. It is the **fundamental reason** why decomposable computation is practical on Kaspa and impractical on Bitcoin.

2. The State Machine Abstraction

2.1 From Smart Contracts to Automata

Michael Sutton proposes the correct mental model for covenant-based applications:

> *"The object I propose as the correct abstraction is a state machine. An automaton."*

In this model:

- **Each state** = active data state + specific constitution (script hash) for the next transition
- **Each transition** = a covenant spend that verifies the next output follows the constitution
- **The control graph** = a finite, predefined set of constitutions/templates
- **Cycles** = allowed (e.g., A → B → A in a chess game)

The key primitive is `become`:

```
become next <- KCC20({<br/> owner_identifier: new_owner_identifier,<br/>
identifier_type: new_identifier_type,<br/> amount: amount,<br/>});
```

This is not a function call. It is a **state transition constraint**: the current contract verifies that the next UTXO is locked under a specific constitution with a specific state.

2.2 The Circularity Problem and Its Solution

A naive state machine fails when script A requires knowing script B's hash, and B requires knowing A's. This is impossible because hashes are computed from the script content.

Michael's solution:

1. **Extract identifiers** from the script itself

2. **Store them as immutable state** (verified never to mutate)

3. **Compile all constitutions** without cross-references

4. **Inject identifiers from the outside** during initialization

5. **Genesis proof** (from covenant id preimage) verifies correct initialization

This bootstrap mechanism is the technical foundation for Axiom's **Identity & Governance Layer (L2)**.

3. The Five Layers: Technical Derivation

3.1 L1: Orchestration

Primitives: `intent.igra`, `agents.igra`, `mcp.igra`

Technical basis: The `become` primitive in Argent.

An agent does not "call a function." It expresses an **intent** that must be satisfied by a valid state machine transition. The orchestration layer answers:

- **Who** is requesting the transition? (`agents.igra` — identity)
- **What** is the desired end state? (`intent.igra` — goal)
- **How** is the transition routed? (`mcp.igra` — Model Context Protocol)

In Argent terms, this maps to the **routing tree** that the compiler builds from the `become` graph. The compiler's main challenge is to inject efficient routing code wherever required.

Connection to Kaskad: Kaskad's covenant-native composable state provides the execution engine for these transitions. Axiom's orchestration layer provides the semantic framework that determines which transitions are economically meaningful.

Why `.igra`? Because Igra provides the semantic namespace for agent-native identifiers. A Kaspa address is a cryptographic primitive; an `.igra` name is an economic primitive.

3.2 L2: Identity & Governance

Primitives: `auth.igra`, `identity.igra`, `governance.igra`

Technical basis: The bootstrap mechanism and covenant IDs.

Every covenant instance has two identifiers:

1. **Script hash** (constitution) — what rules it follows

2. **Covenant ID** (lineage) — which initialization chain it belongs to

The covenant ID prevents the "fake UTXO" attack: anyone can copy a script and create a P2SH address, but they cannot forge the lineage label that proves valid initialization.

In Axiom, this extends to:

- `auth.igra`: Scoped, revocable permissions (the `owner_sig` in Argent's `transfer`)
- `identity.igra`: Verifiable agent identity (pubkey or covenant ID as `identifier_type`)
- `governance.igra`: Mutable policy parameters that cannot change without consensus

Critical insight: The `identifier_type` field in Argent's KCC20 example (0x00 = pubkey, 0x02 = covenant ID) is the germ of the entire Identity Layer. An agent can own an asset directly (pubkey) or indirectly (through another covenant's control).

3.3 L3: Data & Truth

Primitives: `data.igra`, `oracle.igra`, `logs.igra`

Technical basis: The `observes` primitive and `validateOutputStateWithTemplate`.

Argent introduces a revolutionary concept for UTXO systems:

```
observes asset by kcc20_covid {<br/> inputs { proxy: MinterProxy; }<br/> outputs {
proxy: MinterProxy; recipient: KCC20; }<br/>}
```

This allows a covenant in one application to **observe and validate** a transition in another application without owning it.

This is not cross-contract messaging. It is **cross-covenant verification**:

- The observer reads the sibling input's state (`readInputStateWithTemplate`)
- The observer validates the output's constitution (`validateOutputStateWithTemplate`)
- The observer enforces that the transition follows its business logic

For the Agent Economy, this means:

- **Oracle.igra:** External truth enters through observed covenant lanes
- **Data.igra:** Structured data flows between agents via standardized state schemas
- **Logs.igra:** Every transition leaves an immutable, prunable audit trail

Powered by Kaspa Native + Dune: The BlockDAG's high frequency makes oracle updates practical in real-time. Dune provides the indexing layer for `logs.igra` queries.

3.4 L4: Economic Layer

Primitives: `pay.igra`, `escrow.igra`, `settle.igra`, `supply.igra`, `borrow.igra`, `repay.igra`

Technical basis: Multi-contract flows + BlockDAG instant settlement.

This is where Axiom becomes concrete. Consider Agentic Pay:

```
intent.igra → auth.igra → data.igra → escrow.igra → pay.igra → settle.igra →
logs.igra<br/> ↓ ↓ ↓ ↓ ↓ ↓ ↓<br/> Goal Verify Fetch Lock Transfer Confirm Audit<br/>
stated identity price funds value receipt trail
```

Each arrow is a `become` transition. Each box is a covenant in the state machine.

Why `settle.igra` uses BlockDAG finality:

- Sub-second confirmation means an agent can verify settlement before initiating the next action
- No rollbacks, no ambiguity — the economic state converges immediately
- This enables **composable agent workflows**: an agent can chain multiple economic actions in a single block

The escrow primitive: In Argent's minter_proxy example, the `MinterProxy` is a form of escrow — funds are locked under a constitution that requires the controller's approval to release. Axiom generalizes this to arbitrary escrow patterns: time-locked, condition-locked, multi-sig, oracle-locked.

3.5 L5: Infrastructure

Primitives: `kskd.igra`, `custody.igra`, `kurrent.igra`

Technical basis: Kaspa's native token (KAS), Igra's bridge infrastructure, and UTXO custody patterns.

This layer provides the "plumbing" that makes the upper layers practical:

- `kskd.igra`: Native Kaspa denomination and fee handling
- `custody.igra`: Multi-sig and cold-storage patterns for agent treasuries
- `kurrent.igra`: Real-time exchange rate and cross-chain settlement

Why this matters: Agents need to pay fees in KAS, but conduct commerce in arbitrary tokens (KCC20, Igra-native assets). The infrastructure layer handles the denomination switching transparently.

4. The 17 Primitives: Complete Taxonomy

#	Primitive	Layer	Function	Argent/Silverscript Equivalent
1	`intent.igra`	L1	Express economic goal	`entry` function signature
2	`agents.igra`	L1	Agent registry & discovery	Actor definitions
3	`mcp.igra`	L1	Inter-agent communication	Multi-file app composition
4	`auth.igra`	L2	Scoped permissions	`sig_verify`, `op_cov_input_count`
5	`identity.igra`	L2	Verifiable agent ID	`identifier_type` (0x00/0x02)
6	`governance.igra`	L2	Mutable policy	Immutable bootstrap state
7	`data.igra`	L3	Structured state	`state` declarations
8	`oracle.igra`	L3	External truth	`observes` with external covenant
9	`logs.igra`	L3	Audit trail	UTXO history (prunable)

10	`pay.igra`	L4	Value transfer	`transfer` entry
11	`escrow.igra`	L4	Conditional locking	`MinterProxy` pattern
12	`settle.igra`	L4	Final confirmation	`become` with BlockDAG finality
13	`supply.igra`	L4	Liquidity provision	Mint/burn patterns
14	`borrow.igra`	L4	Collateralized debt	Multi-covenant loan flows
15	`repay.igra`	L4	Debt settlement	Reverse mint/burn
16	`kskd.igra`	L5	Native fee handling	KAS denomination
17	`custody.igra`	L5	Treasury management	Multi-sig covenant patterns
18	`kurrent.igra`	L5	Real-time exchange	Cross-covenant rate feeds

5. Case Study: Agentic Pay

5.1 The Scenario

An AI agent receives a spending objective: "Purchase 100 units of compute from Provider X, maximum budget 50 KAS."

5.2 The Flow (Mapped to Axiom)

Step 1: Intent Formation (L1)

```
intent.igra: {<br/> action: "purchase",<br/> resource: "compute",<br/> quantity: 100,<br/> max_budget: 50 KAS,<br/> provider: "provider-x.igra"<br/>}
```

Step 2: Identity Verification (L2)

```
auth.igra: {<br/> agent_id: "buyer-agent.igra",<br/> permission: "spend_up_to_50_KAS",<br/> signature: <agent_private_key_sig><br/>}
```

The agent proves it has the authority to spend from its treasury covenant.

Step 3: Price Discovery (L3)

```
data.igra: {<br/> oracle_source: "compute-market.igra",<br/> current_rate: "0.45 KAS/unit",<br/> total_cost: "45 KAS"<br/>}
```

The agent queries an oracle covenant that observes the compute marketplace.

Step 4: Escrow (L4)

```
escrow.igra: {<br/> amount: 45 KAS,<br/> condition:
"delivery_of_100_compute_units",<br/> timeout: "3600_blocks"<br/>}
```

Funds are locked in a covenant that releases to the provider only upon delivery proof, or back to the buyer after timeout.

Step 5: Payment (L4)

```
pay.igra: {<br/> from: "buyer-treasury.igra",<br/> to: "escrow.igra",<br/> amount:
45 KAS<br/>}
```

Step 6: Settlement (L4)

```
settle.igra: {<br/> delivery_proof: <provider_signature>,<br/> escrow_release:
"provider-x.igra",<br/> receipt: <tx_hash><br/>}
```

Upon delivery confirmation, the escrow covenant `becomes` a payment to the provider. The BlockDAG confirms this in sub-second finality.

Step 7: Audit (L3)

```
logs.igra: {<br/> tx_hash: <hash>,<br/> intent: "purchase_compute",<br/> outcome:
"success",<br/> cost: 45 KAS,<br/> timestamp: <block_timestamp><br/>}
```

5.3 Why This Is Not a Smart Contract

In Ethereum, this would be a single monolithic contract with global state. In Axiom/Kaspa:

- Each step is a **separate covenant** in a state machine
- The agent **orchestrates** the flow by constructing the correct `become` chain
- **No global lock:** escrow, payment, and settlement happen in parallel where possible
- **Prunable:** old intent and auth states disappear after settlement
- **Verifiable:** any observer can trace the full flow through covenant IDs

6. Reproducibility and Integration Depth

6.1 What Makes This Framework Difficult to Reproduce

This document is not abstract philosophy. Every layer maps to a specific Kaspa primitive:

- **L1** → Argent's compiler and routing tree

- **L2** → Covenant IDs and bootstrap mechanism
- **L3** → `observes` and `validateOutputStateWithTemplate`
- **L4** → BlockDAG instant settlement + multi-contract flows
- **L5** → Native KAS fees + Igra bridge infrastructure

Reproducing this framework without equivalent integration depth would require:

- Understanding GHOSTDAG's convergence properties
- Mapping Argent's compiler internals to economic primitives
- Demonstrating equivalent domain expertise across cryptography, compiler design, distributed systems, and economics

This becomes difficult to reproduce without having followed the ecosystem's evolution from the conceptual stage through to concrete implementation.

6.2 Early Semantic Consistency

The `.igra` namespace provides early semantic consistency in naming. While anyone can write a whitepaper, establishing naming conventions before infrastructure exists requires both foresight and sustained presence in the ecosystem.

Domains registered:

- `agentpay.igra` — Early product reference
- `agenticpay.igra` — Framework reference

These are not claims of ownership. They are **early semantic anchors** that may influence how the emerging routing layer is discussed as the ecosystem matures.

6.3 The Hypothesis

> **"Multi-contract flows + Kaspa's high-frequency DAG start to look like a practical model for decomposable computation."* — Michael Sutton*

Axiom extends this hypothesis:

> ***"If decomposable computation is practical, then intent-native commerce is inevitable. Early semantic consistency in naming may influence how the emerging routing layer is discussed."***

This is not a claim of control. It is a **research hypothesis** about how shared vocabulary emerges in technical ecosystems.

7. Acknowledgments & Positioning

This framework is an independent research project by **Andreas** (pay.igra). It is not affiliated with Kaspa Foundation, Igra Labs, Kaskad, or any core development team.

Technical debts:

- Michael Sutton, for Argent, Silverscript, and the state machine abstraction
- The GHOSTDAG research team (Yonatan Sompolinsky et al.), for the convergence engine
- The Igra team, for the semantic namespace infrastructure
- The Kaskad team, for covenant-native composable state and the technical foundation of parallel, non-interfering economic flows
- The Kaspa community, for proving that fair-launch PoW can evolve

Domains secured:

- `agentpay.igra` — Early product reference
- `agenticpay.igra` — Framework reference

Contact: Via Igra Name Service or Kaspa Core R&D; channels.

8. Epilogue: The Agent Economy Stack

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/
>■ AGENT ECONOMY ■<br/>■ Autonomous agents conducting commerce ■<br/>■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ L1:
ORCHESTRATION ■ intent.igra | agents.igra | mcp.igra ■<br/>■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ L2: IDENTITY ■
auth.igra | identity.igra | governance.igra ■<br/>■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ L3: DATA & TRUTH ■ data.igra
| oracle.igra | logs.igra ■<br/>■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ L4: ECONOMIC ■ pay.igra | escrow.igra |
settle.igra ■<br/>■ LAYER ■ supply.igra | borrow.igra | repay.igra ■<br/>■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ L5:
INFRASTRUCTURE ■ kskd.igra | custody.igra | kurrent.igra ■<br/>■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■<br/>■ KASPA BASE LAYER
■ GHOSTDAG | BlockDAG | PoW | Covenants ■<br/>■ ■ Argent DSL | Silverscript |
Kaskad ■<br/>■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■■■■■
```

Built on Axioms. Designed for Autonomous Commerce.

"The Internet was built for humans. The next economy will be built for agents."

End of Document